ILLUMATRIX

FINAL MASTER BLUEPRINT

AI Content Workspace • Product Logic • UX/HUD Design • Architecture • Build & QA Plan

A single source of truth for rebuilding the product correctly in Google AI Studio

CORE PRODUCT PROMISE

Save a source story → generate a configurable script → review and approve it → generate scene-by-scene Google Flow prompts and narration → create two thumbnail prompts → optionally create a metadata/SEO package → review and export. ILLUMATRIX prepares the production package; it does NOT generate videos itself.

Design direction: Obsidian Cyberpunk HUD / Futuristic FUI • Implementation target: React + TypeScript + Vite + Tailwind CSS

Prepared from the project conversation and the supplied original ILLUMATRIX requirements. Any unresolved source conflict is explicitly called out rather than silently guessed.

DOCUMENT MAP

1. Executive summary and non-negotiable product rules
2. What ILLUMATRIX is—and is not
3. Core end-to-end user journey
4. Product configuration and decision logic
5. Source Story Library
6. Script Studio and script-generation intelligence
7. Script review, approval, versions and stale-output rules
8. Scene Prompt Studio for Google Flow
9. Visual style engine and style-only regeneration
10. Audio/narration and continuity system
11. Thumbnail Studio
12. SEO Package and Review & Export
13. API Switcher, quota-aware execution and job queue
14. Cyberpunk HUD UI/UX specification
15. React/TypeScript architecture and data model
16. Error handling, safety, security and persistence
17. Acceptance tests and definition of done
18. Build sequence for Google AI Studio
19. Master build prompt to start the new build
20. Phase prompts for controlled implementation
21. Open decisions and implementation checklist

1. EXECUTIVE SUMMARY & NON-NEGOTIABLE RULES

ILLUMATRIX is an AI-assisted content-preparation workspace for creators who start from their own source story and want a coherent, high-retention script plus ready-to-copy production prompts for Google Flow and thumbnail-image tools. It is not an in-app video renderer.

Non-negotiable product rules

- Build around the user's real content workflow—not around a generic dashboard, API manager, or decorative roadmap.
- Preserve the existing project only if intentionally reusing it; for a clean rebuild, follow this blueprint as the single product source of truth.
- Do not implement video generation inside ILLUMATRIX. Generate prompts and production packages that the user copies into Google Flow or an external image generator.
- Every downstream stage must use the user's selected format, duration, content mode, language, approved script version, and visual style where applicable.
- Never silently overwrite source stories, approved scripts, scene prompts, narration, or manual edits. Use versioning, stale indicators, and confirmation dialogs.
- Do not claim content will go viral. Optimize for hook, clarity, curiosity, pacing, retention, and payoff without promising results.
- Treat free quota as a design constraint, not as a promise that unlimited AI calls are possible. Provide graceful handling when quota is exhausted.
- API Switcher must be connected to actual AI requests. It is not merely a status card or dashboard feature.
- Do not fake live telemetry, quota numbers, latency, provider health, or job progress. If a value is unavailable, label it Unknown / Not reported.
- Each feature must be genuinely functional, tested, and connected to the workflow before it is marked complete.

Original duration conflict — must be resolved visibly

The supplied source requirements mention a minimum 4-minute horizontal 16:9 long video in one section, but also mention a minimum 3-minute long-form script later. This blueprint does not silently choose between them. The implementation should make the minimum duration configurable or mark it as a product decision. Until the owner confirms, show the conflict in the build checklist and do not present a guessed value as final.

Do not proceed by building unrelated UI first. Establish the core workflow and data contracts, then style it with the HUD design system.

2. WHAT ILLUMATRIX IS — AND IS NOT

It is

- A source-story library that preserves text and line breaks.
- A script engine driven by user-selected video format, duration, mode, and language.
- A script-review and approval area that protects user edits.
- A scene-based Google Flow prompt builder with complete, independently copyable scene packages.
- A style engine where visual style can change without changing story meaning or narration.
- A thumbnail prompt studio that produces two separate detailed image prompts.
- An optional SEO/metadata package stage and final review/export stage, as requested in the new HUD workflow.
- A quota-aware shared AI execution layer with transparent provider/failover state.

It is not

- Not a video generator or video editor.
- Not a service that guarantees virality, views, retention, or ranking.
- Not an unlimited free AI service. Provider quotas and terms remain external constraints.
- Not a fake monitoring dashboard: status and quota values must come from actual telemetry or be clearly marked unavailable.
- Not a system that changes the approved script when the user only asked to change visual style.

Product priority order

Priority	What must work	Why
P0 — Core	Source → configuration → script → approval	Without this, later stages have no reliable input.
P1 — Production prompts	Scene prompts, narration, continuity, style regeneration	This is the main value delivered to Google Flow.
P2 — Packaging	Thumbnail prompts, SEO package, review/export	Makes the output useful as a complete creator workflow.
P3 — Operations/UI	API switcher, queue, telemetry, HUD polish	Must support real work, never replace the work itself.

3. CORE END-TO-END USER JOURNEY

Primary pipeline

```
01 SETUP → 02 SOURCES → 03 SCRIPT STUDIO → 04 SCENE PROMPTS → 05 SEO PACKAGE → 06 THUMBNAIL STUDIO → 07 REVIEW & EXPORT
```

The left-side stepper uses these seven stages. Internally, the app may also provide dashboard/control-center panels, but the main canvas must always show the current task and the next useful action.

Detailed workflow

- Setup: create/select a project; select intended output format and default project preferences if the app supports project-level defaults.
- Sources: add a source story, preserve line breaks, edit/save, and select the story that will be used for this run.
- Script Studio: choose format, duration, content mode, language, and optional long-form length. Generate a script from the selected source.
- Review: edit the generated script, compare with source if useful, save revisions, and explicitly approve the script version.
- Scene Prompts: map approved script passages into scenes. Each scene has a full Google Flow prompt, narration, duration/ratio, and continuity package.
- SEO Package: create title ideas, description, keywords/tags, and optional chapters only when appropriate. Never invent factual metadata.
- Thumbnail Studio: analyze the approved script and selected mode/ratio, then generate two distinct image prompts.
- Review & Export: validate completeness, show stale-output warnings, let the user copy/download the production package, and preserve project state.

State transitions

Current state	Allowed action	Next state / result
No source selected	Select or create a story	Source selected
Source selected	Configure and generate script	Draft script
Draft script	Edit / save / regenerate with confirmation	Draft version updated
Draft script	Approve script	Approved script version
Approved script	Generate prompts	Prompt set tied to script version
Prompts generated	Edit scene / regenerate one scene	Scene version updated
Prompts generated	Change visual style and regenerate	Visual prompts updated; script/narration preserved
Approved script	Generate thumbnail prompts	Two thumbnail prompt drafts
Any output becomes outdated	Change upstream source/config/script	Show STALE OUTPUT; require review/regeneration decision

4. PRODUCT CONFIGURATION & DECISION LOGIC

Required configuration before script generation

Setting	Choices / rules	Downstream effect
Main video format	9:16 Vertical Shorts OR 16:9 Horizontal Long-form	Controls script structure, duration options, prompt ratio, thumbnail ratio, export metadata.
Short duration	24 / 32 / 40 / 48 seconds only	Exactly 3 / 4 / 5 / 6 scenes respectively; each scene designed for 8 seconds.
Long-form duration	Owner decision needed: original spec conflicts between 3 and 4 min minimum	Long-form script and scene plan must respect selected/configured duration.
Source story	Dropdown/select from saved story library	Defines source content; retain source ID/version.
Content mode	Documentary, Explainer, Breaking News, Horror Story, Funny, Sci-Fi Explainer, Detailed Explainer + extensible list	Controls tone, pacing, structure, factual caution, vocabulary, visual treatment.
Language	Hindi or English	Script and narration must follow selected language; prompts may be English production instructions if helpful.
Visual style	Cinematic default + 12 requested styles	Applies to visual prompts only; must not alter script meaning or narration.
Optional settings	Audience, platform, CTA, tone intensity, fact-check notes if supported	May refine outputs but must never be required to complete the core workflow.

Short duration mapping

Selected duration	Scene count	Scene plan
24 sec	3	0–8s Hook; 8–16s Main idea; 16–24s Ending/payoff
32 sec	4	Hook; setup/context; main reveal/explanation; ending/payoff
40 sec	5	Hook; context; development; key reveal/detail; ending/payoff
48 sec	6	Hook; context; development 1; development 2; payoff; ending/CTA if appropriate

These are structural guides, not rigid templates for every mode. The model may adapt the middle scenes to the source story, but it must preserve the scene count and keep the opening, development, and payoff functions clear.

Configuration validation

- Do not show Shorts duration options when 16:9 long-form is selected.
- Do not allow 24/32/40/48-second settings to leak into long-form.
- Do not allow a prompt to use a different ratio from the project's main format unless the user intentionally changes the project configuration.
- Store the configuration snapshot with each script version so later outputs can be reproduced.
- Changing format, duration, source story, mode, or language marks dependent outputs stale; it must not silently delete them.

5. SOURCE STORY LIBRARY

Purpose

This is the ground-truth input library. The user supplies a story/article/notes and expects the text to remain saved line by line. Downstream generation must be traceable to a selected saved source.

Required UI

- Create/Add Story action.
- Story title/name field; optional source type, notes, and creation date.
- Large plain-text editor that preserves paragraphs and line breaks.
- Save, edit, duplicate, delete/archive (with confirmation), search/filter, and select story actions.
- Source preview with character/word count and last-saved indicator.
- Selection control in Script Studio that identifies exactly which story version is used.
- Unsaved-change warning before navigation or switching to another story.

Data and behaviour

- Save source text exactly as entered; do not automatically rewrite, summarize, normalize punctuation, or remove blank lines.
- Keep stable story ID and version number or updated timestamp.
- On editing a story after a script was generated, keep the old script intact and mark dependent outputs STALE SOURCE.
- Deleting a story must not silently erase scripts or prompts that were already created from it; use archive/soft-delete or dependency warning.
- Store large source text safely within the chosen persistence architecture; define size limits and show validation errors instead of truncating silently.
- Allow a story to be selected explicitly before generating a script; do not use whichever story happened to be edited most recently unless it is visibly selected.

Acceptance criteria

- Save a story containing multiple paragraphs and blank lines, refresh, and verify exact text remains.
- Create two stories, select the first, and verify script generation references the first story ID/version.
- Edit the source after generation; existing output remains available and visibly stale.
- Navigate away with unsaved edits; the app warns and offers save/discard/cancel.

6. SCRIPT STUDIO & SCRIPT-GENERATION INTELLIGENCE

Purpose

The script engine transforms a selected source into an engaging script suited to the selected mode, format, duration, and language. It should use editorial judgement to improve clarity, hook, pacing, curiosity, and payoff while remaining faithful to the source.

Required controls

- Selected source story dropdown with title and version.
- Main format selector: 9:16 Shorts / 16:9 Long-form.
- Duration selector that changes dynamically based on format.
- Content mode selector.
- Language selector: Hindi / English.
- Generate Script button with clear loading/error states.
- Editable script output; for Shorts, scene-by-scene segmentation; for long-form, sections/beats and estimated duration.
- Regenerate action with a confirmation if it could replace edits; ideally offer regenerate selected section vs full script.
- Save Draft and Approve Script actions.

Mode behaviour

Mode	Script traits to request
Documentary	Evidence-aware, measured, informative, narrative progression; avoid unsupported claims.
Explainer	Plain-language explanation, clear sequence, analogy when useful, concept-to-takeaway.
Breaking News	Lead with confirmed development, distinguish confirmed facts from unknowns, time/context labels if supplied; never fabricate current facts.
Horror Story	Atmosphere, tension escalation, sensory details, controlled reveals, satisfying payoff; mark fiction if source is fictional.
Funny	Setup, escalation, timing, contrast, punchline; do not force jokes that distort factual source.
Sci-Fi Explainer	Speculative/futuristic framing clearly separated from real facts; coherent concepts and vivid examples.
Detailed Explainer	More context and explanatory depth; structured sequence and clear definitions.
Extensible modes	Mode definitions should be data-driven so new modes can be added without rewriting core logic.

Script engine rules

- Read and reason over the full selected source before drafting; identify core message, key facts/events, causal order, and audience-relevant detail.
- Generate a hook that opens a curiosity loop but does not mislead about what the source supports.
- Maintain narrative coherence from hook to main content to ending/payoff.
- Do not make fabricated statistics, quotes, names, dates, scientific claims, or news developments.
- Preserve essential facts and distinguish creative framing from factual content.
- Respect selected language. Hindi means natural Hindi, not an accidental English output with a Hindi title; English means natural English.
- Match mode-specific tone, but avoid parodying serious subjects or turning breaking news into sensational misinformation.
- Use target duration as a pacing constraint; include estimated narration word count or speaking-time estimate when useful.

- Output explicit scene boundaries for Shorts, each with a scene purpose and narration text.
- For long-form, output title/hook, intro, logically ordered sections, transitions, conclusion, and optional CTA. The minimum length remains blocked on the 3-vs-4-minute decision.

Do not promise virality

Use phrasing such as “optimize for retention and curiosity” rather than “make viral content.” Performance depends on the audience, platform, delivery, editing, timing, and many factors outside the app.

7. SCRIPT REVIEW, APPROVAL, VERSIONS & STALE OUTPUTS

Review screen

- Show source story and generated script in distinct panels or tabs.
- Allow editing without leaving the workflow.
- Display selected format, duration, mode, language, source ID/version, and script version.
- Show word count and estimated duration where possible.
- Provide Save Draft, Approve Script, and Regenerate actions.
- Require explicit approval before scene prompt generation.

Versioning and user-edit protection

- Every generation creates a new draft version or a deliberate revision; it must not overwrite the approved version invisibly.
- Store user-edited content separately from model-generated initial output or keep a clear version history.
- Before regenerate-all, show a modal describing what may be replaced.
- Regenerate a section only when requested; preserve all other sections.
- Approving a new script version should mark prompts and thumbnails tied to the prior version as stale, not delete them.
- Prompt Studio must store the exact approved script version ID it used.

Stale-output matrix

Upstream change	Mark stale	Must not change automatically
Source story edited	Script, prompts, thumbnails, SEO package	Existing script and all downstream manual edits
Source story selection changed	Script and all downstream outputs	Previous story's saved outputs
Format/duration/mode/language changed	Script and downstream outputs	Saved output versions
Approved script changed	Prompts, thumbnails, SEO package	New approved script text itself
Visual style changed	Visual prompts only	Approved script, narration, source story
Narration manually edited	Scene package and any audio-related export	Visual prompts unless the user asks to sync them
SEO title/description edited	SEO package only	Script, prompts, thumbnail prompts

8. SCENE PROMPT STUDIO FOR GOOGLE FLOW

Purpose and hard boundary

Prompt Studio produces detailed, independently usable prompts for external video generation. The app does not call Google Flow to render videos and does not pretend that a prompt is a generated video.

Scene package schema

- Stable scene ID and scene number.
- Scene title and narrative purpose (Hook / Context / Main idea / Reveal / Ending etc.).
- Approved script passage linked to the scene.
- Target duration (8 seconds for each Shorts scene; long-form scene duration based on scene plan).
- Aspect ratio (9:16 or 16:9) written explicitly in the prompt.
- Visual prompt: subject, action, environment, composition, framing, camera motion, lighting, texture, atmosphere, motion, transition/end frame, and negative constraints where useful.
- Narration/voice-over text in the selected language, separate from the visual prompt.
- Continuity instructions: characters, props, location, palette, lighting, camera language, narrator delivery, music mood.
- Optional sound-design notes and on-screen text instructions.
- Generated/edited status, version, timestamp, and source approved-script version.

Prompt-generation workflow

- Verify an approved script exists.
- Split script according to the saved scene structure; preserve the exact Shorts scene count.
- Map every scene to the corresponding script passage; avoid generic filler visuals.
- Generate the shared continuity bible first, then include relevant continuity instructions inside each scene prompt so copied prompts remain self-contained.
- Generate narration separately; do not assume Google Flow will synthesize the intended voice from visual prompt text.
- Show scene cards with edit fields and complete package copy action.
- Let the user regenerate one scene or all scenes. Single-scene regeneration must not change other scenes.
- Before regenerate-all, show a modal and protect manual edits unless the user explicitly confirms replacement.
- Persist each scene package and restore it after refresh.

Complete scene package copy format

```
SCENE 02 — [TITLE]\nFORMAT: 9:16 vertical | DURATION: 8 seconds\nSCRIPT BEAT: [approved passage]\nVISUAL PROMPT: [full detailed prompt]\nCONTINUITY: [character/location/style/lighting/music cues]\nNARRATION (Hindi/English): [voice-over text]\nSOUND DESIGN: [optional guidance]
```

Prompt quality checklist

- Every scene has a clear visual subject and an action that can plausibly happen in the target duration.
- Camera movement is purposeful, not an arbitrary list of cinematic terms.
- Lighting, tone, and color are consistent with style and continuity.
- The visual communicates the narration without contradictory imagery.
- Prompt explicitly states aspect ratio and duration.
- Scene ending leaves a usable transition or visual endpoint where appropriate.
- Text inside the generated video is avoided unless needed; exact on-screen text is specified if necessary.
- Each scene works when copied alone and does not depend on the external model seeing the previous prompt.

9. VISUAL STYLE ENGINE & STYLE-ONLY REGENERATION

Default style is Cinematic. The user can choose a different style and regenerate visual prompts. The content/story/narration remain the same unless the user explicitly asks to change them.

Style option	Prompt treatment
Cinematic (default)	Film-like composition, motivated camera movement, realistic or selected-subject rendering, controlled lighting, coherent color grade.
Paper Craft / Stop Motion Layered Diorama	Layered paper cutouts, visible paper texture, handcrafted depth, gentle stop-motion movement.
Claymation / Aardman-inspired clay aesthetic	Expressive clay characters, tactile fingerprints, playful physical movement, handcrafted stop-motion look. Avoid claiming official studio affiliation.
3D animated family-film aesthetic	Expressive 3D forms, appealing materials, rich lighting and cinematic movement. Avoid presenting it as an official proprietary studio style.
Ghibli-inspired hand-drawn fantasy anime	Lush hand-drawn environments, soft natural light, wind and environmental motion, dreamy atmosphere; describe traits rather than claiming official affiliation.
90s Retro Anime Cel Shading	Bold cel shading, limited animation feel, vibrant palette, dynamic action lines.
Watercolor Illustration	Blended pigment, visible paper grain, soft edges, flowing painted movement.
Ink Wash / Sumi-e	Expressive ink strokes, minimalist negative space, monochrome wash, poetic motion.
Flat Vector Illustration	Clean shapes, graphic color blocks, smooth 2D motion, controlled transitions.
Isometric 3D Render	Isometric camera, detailed miniature environments, precise geometry, gentle camera orbit.
Pixel Art 8-Bit	Pixel-grid discipline, limited palette, crisp sprite movement, retro game aesthetic.
Comic Book / Graphic Novel Panels	Bold ink lines, halftone textures, panel-like transitions, dynamic pans/zooms.
Noir Black and White Sketch	High-contrast monochrome, dramatic shadows, sketch texture, restrained movement.

Style change algorithm

- Keep approved script version, scene order, scene purpose, duration, aspect ratio, narration text, and factual content unchanged.
- Replace only style-specific visual descriptors, materials, motion language, lighting, palette, texture, and transition approach.
- Update continuity bible's visual-style section and propagate the updated style into every regenerated visual prompt.
- Keep narrator tone/music mood stable unless the user explicitly asks to change audio direction.
- Mark only visual prompts as updated; do not mark script or narration as regenerated.
- Allow style to be applied to one scene or all scenes; if all scenes are selected, confirm before overwriting manual prompt edits.
- Do not make proprietary studio affiliation claims; use descriptive visual characteristics.

10. AUDIO, NARRATION & CONTINUITY SYSTEM

Narration rules

- Every scene includes separate TTS-ready narration in the selected language.
- Use consistent narrator identity descriptors: perceived voice quality, pace, energy, pronunciation style, and emotional intensity.
- Maintain a stable narration delivery profile across scenes; changes should be intentional and documented.
- Do not imply that the app generates actual audio unless an audio-generation integration is separately implemented.
- Keep narration text editable independently from visual prompts.
- Where practical, estimate speaking time and warn if narration is too long for the scene duration.

Audio/visual continuity bible

- Voice: language, narrator profile, pace, emotional range, pronunciation notes.
- Music: genre/mood, tempo feel, intensity curve, instruments/textures, beginning/end behaviour.
- Sound design: ambience, key effects, loudness hierarchy, avoid competing with narration.
- Visual: palette, lighting, camera movement grammar, recurring subjects, locations, props, atmosphere.
- Each copied scene prompt contains the relevant subset of the continuity bible. Do not assume external video generations share hidden memory.
- Because separate generations may vary, phrase continuity as instructions rather than a guarantee of identical outputs.

Consistency check

Before export, compare all scenes for style, ratio, narrator language, music mood, and continuity terms. Flag mismatches. Do not automatically rewrite manually edited scenes; provide a review action.

11. THUMBNAIL STUDIO

Inputs

- Approved script version and selected source story context.
- Selected format: 9:16 or 16:9.
- Selected content mode and language.
- Optional visual style or separate thumbnail visual direction.
- Title/curiosity angle derived from the script without false claims.

Required output

Generate two distinct, detailed image prompts—not two copies with minor wording changes. The user will paste these prompts into ChatGPT, Gemini, or another image generator. ILLUMATRIX is not required to render the thumbnail itself.

- Prompt A and Prompt B should use meaningfully different compositions or visual hooks.
- Each prompt includes aspect ratio and intended framing.
- Describe main subject, facial expression or emotional cue if appropriate, environment, foreground/background, lighting, color palette, depth, realism or illustration treatment, camera/viewpoint, and negative constraints.
- Recommend concise title overlay text or a title treatment in a scientific/curiosity-driven visual style where appropriate.
- Keep text overlay suggestions legible and limited; clearly state exact words and placement.
- Reflect the selected content mode; for example, horror should not look like a generic tech explainer.
- Use the selected format: 9:16 composition for Shorts, 16:9 for long-form.
- Make both prompts faithful to the approved script and avoid fabricated visual claims or misleading clickbait.
- Provide separate Copy Prompt A, Copy Prompt B, and Copy All actions.

Regeneration and stale logic

- If script version changes, mark thumbnail prompts stale.
- If only the visual style of video prompts changes, thumbnails need not become stale unless the thumbnail style is explicitly linked to video style.
- Regenerate with confirmation if the user has manually edited prompt text.
- Persist both prompts and their relationship to the script version.

12. SEO PACKAGE & REVIEW / EXPORT

SEO Package (new HUD workflow stage)

This stage is an additional packaging feature in the requested UI workflow. It must support the core story-to-prompt pipeline rather than displace it.

- Generate several title options based on the approved script and selected mode/language.
- Generate a concise description/summary, relevant keyword/tag suggestions, and a CTA if appropriate.
- Offer chapter/timestamp suggestions only when a long-form script has enough structure; do not invent exact final timestamps before a final edit exists.
- Keep metadata editable and versioned.
- Do not promise search ranking, views, or virality.
- Mark the SEO package stale if the approved script or language/mode changes.

Review & Export

- Show a completeness checklist: source selected, script approved, prompts present for all required scenes, narration present, style/ratio consistent, thumbnails available, optional SEO package status.
- Show explicit STALE OUTPUT badges with the upstream change reason.
- Allow copying one scene, all scenes, narration only, thumbnails, SEO metadata, or the full project package.
- Provide export as a readable text/Markdown package and, if implemented, JSON for structured backup.
- Never export API secrets, internal credentials, or private provider configuration.
- Warn about missing required outputs without blocking export of otherwise usable work unless a dependency is essential.

13. API SWITCHER, QUOTA-AWARE EXECUTION & JOB QUEUE

Purpose

The API Switcher is an operational service used by real AI requests. Its job is to choose an eligible configured provider connection, handle quota/provider failures safely, record truthful status, and preserve user work.

Shared execution layer

```
UI action → AI Job Service → Provider/Connection Selector → Request Adapter → Provider API → Response Validation → Persist Output
```

Script Studio, Prompt Studio, style regeneration, thumbnail prompts, and SEO package generation should call the same execution layer with task-specific request definitions. Avoid separate conflicting switchers inside each feature.

Connection selection and failover

- Maintain a registry of configured provider connections, capability, enabled state, credential reference, and last-known status.
- Keep credentials in server-side secrets or a supported secure runtime; never ship secret keys in browser bundles.
- Classify failures: rate limit, quota exhausted, transient network/provider error, invalid credentials, permission/configuration error, malformed request, and unknown error.
- Fail over only for errors where another eligible connection may help. Invalid request/content errors should not trigger endless switching.
- Use bounded retry counts, exponential backoff where appropriate, cooldown for rate-limited connections, and loop prevention.
- Track which connection was attempted, outcome category, and task/job ID without logging secrets or sensitive prompt content unnecessarily.
- Persist successful output before reporting completion to the UI.
- On total failure, preserve current drafts and edits and show a clear retry/recovery message.

Critical quota truth

Different API keys do not automatically mean different quota pools. Keys tied to the same project, billing account, model, or quota scope may share limits. The app must not claim to bypass provider limits or create unlimited free capacity.

Quota monitor UI

- Show RPM/TPM/RPD only when the provider actually exposes these metrics or the app has reliable measurements.
- Distinguish provider-reported quota from locally observed request counts and estimates.
- Show last-updated timestamp and metric source.
- If data is unavailable, display “Not reported” rather than fabricated bars.
- Do not label a provider “Gemini Ultra/Pro” unless that exact configured model/connection exists; display actual model/provider labels.

Job queue

- Job states: queued, running, succeeded, failed, waiting for cooldown, retrying, cancelled.
- Each job has stable ID, task type, source/output version links, timestamps, attempt count, and user-visible status.
- Allow cancellation where supported; cancellation must not corrupt saved output.
- On failover, show a concise alert and record the event.
- Do not run multiple duplicate generations because the user clicked repeatedly; use in-flight guards/idempotency where practical.
- Progress percentages should reflect real steps or be labeled as phase-based estimates.

14. CYBERPUNK HUD UI/UX SPECIFICATION

Visual design system

Token	Specification	Usage
Background	#030712 / #000000	App canvas, shell, deep workspace
Surface	#0B0F19	Dark glass cards and panels
Border	#1E293B	1px card edges and dividers
Primary accent	#FFE600 / #EAB308	Critical alerts, selected telemetry labels, primary CTA
Secondary accent	#00F0FF / #06B6D4	Active nodes, selected navigation, radars, charts
Success	#10B981	Online/success states
Error	Use a readable red/pink such as #FB7185	Failed job / invalid state
Typography	JetBrains Mono, Share Tech Mono, Orbitron or safe fallbacks	HUD labels, IDs, telemetry; body text remains readable
Effects	Subtle glow, restrained scanlines, glass panels	Use selectively; readability and performance win over decoration

Layout regions

Top status bar

- Project title ILLUMATRIX and current project name.
- Active workflow stage and overall stage indicator.
- Master Brain status only if a real orchestrator/status exists; otherwise label as local workflow status.
- API node status: online/offline/unknown, active connection label, latency when measured.
- Current job state and last-updated timestamp.
- Responsive compact mode on narrow screens.

Left workflow stepper

- 1 Setup; 2 Sources; 3 Script Studio; 4 Scene Prompts; 5 SEO Package; 6 Thumbnail Studio; 7 Review & Export.
- Show completed/current/blocked/stale states, not just decorative numbered links.
- Disable or explain blocked downstream steps when prerequisites are missing.
- Keep navigation and current draft state stable.

Central workspace

- Primary work area for current task, not a dashboard of unrelated widgets.
- Terminal-style activity stream for meaningful events: story saved, script generation started, scene 2 regenerated, provider failover, export ready.
- Segment cards with stable IDs, duration tags, ratio, status, edit actions, and copy actions.
- Circular radar/progress visualizer may be SVG/CSS animation, but it must reflect real job states and stop when work finishes.
- Use skeleton/loading states and clear error recovery; never fake generated output.

Right Control Center / Quota Monitor

- Provider connection cards showing actual configured provider/model, status, cooldown, and last result.
- Quota bars only for reliable metrics; clearly distinguish measured vs estimated.
- Job queue with state, attempts, failover alerts, and retry action.
- Keep panel collapsible on smaller screens so it does not crowd the editor.

Interaction and accessibility rules

- Primary actions are visually clear; destructive/regenerate-all actions require confirmation.

- All buttons have working actions; no decorative controls that look functional but do nothing.
- Use keyboard focus states, sufficient contrast, visible labels, and accessible modal behaviour.
- Do not use animation to communicate status alone; pair with text/icon.
- Respect reduced-motion preferences; keep scan/radar loops subtle and non-blocking.
- Use responsive layout: three-column desktop; collapsible side panels/tablet; stacked workspace/mobile.
- Show unsaved changes and stale-output badges near the affected artifact.
- Use stable segment IDs for identity; do not key scene cards by array index alone when IDs are available.

15. REACT/TYPESCRIPT ARCHITECTURE & DATA MODEL

Target stack

- React + TypeScript + Vite.
- Tailwind CSS with named design tokens/utilities for accent colors, borders, glow, and spacing.
- Lucide React icons (Radio, Activity, Cpu, Shield, Zap, Database, FileText, Clapperboard, Image, Settings, Copy, RefreshCw, AlertTriangle, CheckCircle).
- Framer Motion for restrained panel/step transitions and active-node animation.
- Use SVG/CSS for the radar visualizer unless a real data visualization requires another library.
- Choose persistence appropriate to the deployment: backend/database preferred for durable multi-session use; local storage may be acceptable only for a prototype and must be clearly scoped.

Suggested domain entities

Entity	Important fields
Project	id, name, createdAt, updatedAt, activeConfigId
SourceStory	id, projectId, title, rawText, version, createdAt, updatedAt, archivedAt
GenerationConfig	id, format, duration, contentMode, language, visualStyle, longFormMinimum
ScriptVersion	id, sourceStoryId, sourceVersion, configSnapshot, draftText, approvedText/status, version, createdAt
ScenePackage	id, scriptVersionId, sceneNumber, scenePurpose, sourcePassage, visualPrompt, narration, continuity, duration, ratio, style, version, editedFlags
ContinuityBible	id, scriptVersionId, visualRules, audioRules, narratorProfile, musicProfile
ThumbnailPromptSet	id, scriptVersionId, promptA, promptB, configSnapshot, status, version
SeoPackage	id, scriptVersionId, titles, description, tags, chapters, status, version
AIJob	id, taskType, inputRefs, status, attempts, providerAttempts, errorCategory, timestamps
ProviderConnection	id, provider, model, secretRef, enabled, capabilities, lastStatus, cooldownUntil
AuditEvent	id, projectId, eventType, artifactRef, timestamp, safeMetadata

Suggested service boundaries

- sourceStoryService: save/read/version/archive story.
- scriptService: validate config, create generation request, save draft, approve version.
- promptStudioService: create scene breakdown, continuity bible, per-scene packages, regenerate scene/all.
- thumbnailService: create and version two thumbnail prompts.
- seoService: metadata package generation.
- aiExecutionService: shared provider selection, retries, failover, job tracking, response validation.
- staleStateService: compute dependency-based stale status.
- exportService: assemble readable production package and optional structured export.

Architectural rule

Components should not call provider APIs directly. They should invoke typed services that use the shared AI execution layer. This keeps quota handling, error handling, observability, and persistence consistent.

16. ERROR HANDLING, SECURITY & PERSISTENCE

User-facing error behaviour

Failure	Expected behaviour
No source selected	Explain that a source story must be selected; keep user on the current step.
No approved script	Block prompt generation with a clear link back to Script Studio.
Quota exhausted	Preserve input; show quota message, failover result, and when retry may be possible if known.
Provider unavailable	Attempt eligible failover within limits; then show retry option.
Invalid credentials	Mark that connection as configuration-required; do not repeatedly retry it.
Output validation failed	Do not save malformed output as complete; offer retry and preserve prior valid version.
Browser refresh	Restore saved project and output state according to the persistence guarantee.
Stale output	Show why stale and offer review/regenerate, never silently replace.
Copy/export failed	Show a clear error and keep content visible for manual copy.

Security and privacy

- Never expose API secrets in client-side code, source maps, console logs, user-visible error text, exports, or audit events.
- Use server-side secret storage where the chosen deployment supports it.
- Validate and limit user input sizes; handle malformed provider responses.
- Do not log full story text or sensitive prompt content unless necessary and disclosed.
- Protect project data according to the selected storage and auth model.
- Use confirmation for destructive actions and preserve recovery paths.

Persistence rules

- Source stories survive refresh and new sessions to the extent promised by the selected storage architecture.
- Script versions retain their source/config references.
- Scene prompts, narration, continuity, thumbnail prompts, SEO package, and user edits persist.
- Store generation status and timestamps so UI can distinguish saved output from transient state.
- Define backup/export path so the user can preserve work independently.

17. ACCEPTANCE TESTS & DEFINITION OF DONE

Core functional tests

- Create, save, edit, and reopen a source story with multiple paragraphs; line breaks remain exact.
- Select one story from multiple saved stories; generated script references the correct source version.
- Generate 24/32/40/48-second Shorts and verify exactly 3/4/5/6 scenes respectively.
- Verify 24-second plan follows Hook → Main Part → Ending.
- Verify 16:9 hides Shorts duration choices and uses the owner-confirmed long-form minimum.
- Switch Hindi/English and verify script and narration language follows selection.
- Test each content mode's tone and structure with a representative fixture.
- Edit and approve a script; downstream prompts are blocked until approval.
- Change source/config/script; dependent artifacts display the correct stale status.
- Generate scene prompts; each includes ratio, duration, visual details, narration, and continuity.
- Copy a single scene package; copied content includes all necessary independent-use details.
- Regenerate one scene; all other scene packages and manual edits remain unchanged.
- Change visual style; script and narration stay identical while visual prompts update.
- Generate two distinct thumbnail prompts for both aspect ratios.
- Generate and edit SEO metadata; it stays tied to the script version.
- Refresh the app; persisted artifacts return intact.

API Switcher tests

- Simulate rate limit, quota exhaustion, transient failure, invalid credentials, and malformed response.
- Verify eligible failover occurs only for appropriate error categories.
- Verify retries are bounded and no endless loops occur.
- Verify same-quota connections are not falsely presented as extra quota.
- Verify secrets are absent from UI, logs, and exports.
- Verify failure preserves user edits and the last valid output.
- Verify telemetry shows Unknown/Not reported when data is unavailable.

UI/UX tests

- All seven workflow steps show current/completed/blocked/stale status correctly.
- Every visible button and menu action works.
- Regenerate-all and destructive actions show confirmation.
- Unsaved edits trigger a warning.
- Responsive layout works at desktop, tablet, and mobile widths.
- Reduced-motion setting is respected and text contrast remains readable.
- Radar/telemetry animations reflect real job state rather than a fake always-running status.

Definition of done

A feature is done only when its UI, state transitions, persistence, error handling, dependency/stale logic, and acceptance tests work together. A rendered screen or mock response alone is not implementation.

18. BUILD SEQUENCE FOR GOOGLE AI STUDIO

Use one phase at a time. After every phase, run the acceptance tests and inspect the actual generated code before moving on. Do not ask the builder to redesign everything repeatedly.

Phase	Deliverable	Gate before next phase
0. Audit & architecture	Inspect current files; identify reusable work, gaps, stack, persistence, API integration, risks.	Written gap report and agreed data contracts.
1. Foundation + HUD shell	React/TS/Vite/Tailwind tokens, responsive shell, top bar, stepper, workspace, control panel, real state labels.	No dead controls; navigation shell works.
2. Source Library + Setup	Story CRUD/versioning, exact text preservation, configuration controls and validation.	Persistence and config rules pass.
3. Script Studio	Mode/language/format/duration-aware generation, editable drafts, approval and versions.	Scene counts, language, approval, stale logic pass.
4. Shared AI execution/API Switcher	Connection registry, error classification, bounded retry/failover, job queue, safe telemetry.	Failure simulation tests pass.
5. Prompt Studio	Continuity bible, per-scene packages, narration, copy actions, persistence, regeneration protection.	Shorts scene mapping and single-scene isolation pass.
6. Style engine	12 styles + cinematic default; visual-only regeneration.	Script/narration unchanged by style change.
7. Thumbnail Studio	Two detailed prompts, format/mode-aware, editable and versioned.	Two distinct outputs and stale rules pass.
8. SEO + Review/Export	Metadata package, completeness checks, stale badges, copy/export.	Export accurate and secrets excluded.
9. Hardening	Responsive QA, accessibility, persistence, error states, tests, final audit.	All critical acceptance tests pass.

The API Switcher is integrated early enough to be tested but must not hijack the product design. The content workflow remains the primary user experience.

19. MASTER BUILD PROMPT — START THE NEW BUILD

```
You are the Lead Product Engineer, Senior Frontend Architect, AI Workflow Engineer, and UI/UX Designer for ILLUMATRIX.

Treat the attached ILLUMATRIX Final Master Blueprint as the single source of truth. First read it fully. Do not invent a different product, simplify away core logic, or prioritize a generic dashboard over the content workflow.

PRODUCT PURPOSE:
ILLUMATRIX accepts user-saved source stories and creates configurable scripts, approved script versions, detailed scene-by-scene Google Flow prompts with narration and continuity, two thumbnail-image prompts, optional SEO metadata, and a final export package. It does not generate videos inside the app.

TECH TARGET:
React + TypeScript + Vite + Tailwind CSS, Lucide React, and Framer Motion. Use the specified Obsidian Cyberpunk HUD visual system. Adapt to the existing codebase only if a deliberate reuse decision is documented.

CORE WORKFLOW:
Setup → Sources → Script Studio → Scene Prompts → SEO Package → Thumbnail Studio → Review & Export.

NON-NEGOTIABLE LOGIC:
- Save source stories exactly, preserving line breaks.
- Script Studio must use selected source story, selected format, duration, mode, and language.
- Shorts are 9:16 and only 24/32/40/48 seconds; these map to exactly 3/4/5/6 scenes of 8 seconds each.
- For 24 seconds use 8-second hook, 8-second main content, 8-second ending/payoff.
- Long-form is 16:9. The source spec conflicts between 3-minute and 4-minute minimum; do not silently resolve it. Surface it as a configurable/owner-confirmation decision.
- Content modes include Documentary, Explainer, Breaking News, Horror Story, Funny, Sci-Fi Explainer, Detailed Explainer, and an extensible registry.
- Languages are Hindi and English.
- Require script review and explicit approval before Prompt Studio.
- Scene prompts must be self-contained when copied individually and include ratio, duration, detailed visual direction, narration, and relevant continuity.
- Default visual style is Cinematic. Support the 12 styles listed in the blueprint. Style-only regeneration must preserve script, scene meaning, and narration.
- Thumbnail Studio generates two distinct detailed prompts, not images.
- Add editable SEO metadata and final review/export.
- Protect manual edits, use versioning, and show explicit stale-output warnings.
- Integrate all AI tasks with one shared API execution layer. Handle quota/rate-limit/provider errors, eligible failover, bounded retries, secure secrets, and honest telemetry.
- Never claim unlimited free quota or guaranteed virality.
- Do not fabricate provider status, latency, or quota values.
- All visible controls must work.

DESIGN:
Obsidian #030712/#000000 background; surfaces #0B0F19; border #1E293B; cyberpunk yellow #FFE600/#EAB308; electric cyan #00F0FF/#06B6D4; success emerald #10B981. Use monospace/futuristic headers, readable body text, subtle glow, top status bar, left 7-step workflow, central workspace, right Control Center/Quota Monitor, segment cards, and a radar/progress visualizer tied to actual job state.

WORK METHOD:
1. Audit the repository and list what exists, what is missing, and what must be preserved.
2. Propose data models and service boundaries before broad changes.
3. Implement one phase at a time; do not rebuild everything repeatedly.
4. After each phase, run tests and report actual results.
5. Do not mark mock UI as functional implementation.
6. Preserve existing work and user edits; use confirmation before destructive regeneration.
7. If a requirement is ambiguous, flag it and use a configurable setting rather than guessing.
8. Finish with changed files, tested behaviours, known limitations, and manual setup required.

Start with Phase 0 only: inspect the existing codebase, report the gap analysis, identify the persistence/API environment, and propose a safe implementation order. Do not begin a large rewrite until the audit is complete.
```

How to use it

- Start a new Google AI Studio project if you truly want a clean rebuild; attach/paste this blueprint and the master prompt.
- If keeping the existing repository, use the same prompt but insist on Phase 0 audit before any rewrite.
- Do not paste every phase at once and let the builder generate a giant untested code dump.
- After Phase 0, proceed phase-by-phase using the prompts below and the relevant acceptance tests.

20. PHASE PROMPTS FOR CONTROLLED IMPLEMENTATION

Phase 0 — Audit only

```
Read the ILLUMATRIX Final Master Blueprint. Inspect the existing project and return a gap analysis: current stack, routes/components, data models, persistence, provider/API integration, working features, broken or missing features, risks, and files to preserve. Confirm the 3-vs-4-minute long-form ambiguity. Do not rewrite code yet. Propose a phase plan and test plan.
```

Phase 1 — HUD shell

```
Implement only the responsive ILLUMATRIX HUD shell according to the blueprint: top status bar, seven-step workflow stepper, central task workspace, right Control Center, shared design tokens, reusable panels, status badges, modals, and loading/error/empty states. Use React/TypeScript/Vite/Tailwind, Lucide, and restrained Framer Motion. No fake metrics and no dead controls. Wire navigation and real stage state.
```

Phase 2 — Sources + setup

```
Implement Source Story Library and configuration. Preserve source text and line breaks exactly. Add save/edit/select/archive/version handling and unsaved-change protection. Add format, duration, mode, and language controls with correct conditional rules. Persist state and test refresh. Do not implement script generation in this phase unless needed for a minimal integration test.
```

Phase 3 — Script Studio

```
Implement source-aware script generation using selected config. Include mode registry, Hindi/English, exact Shorts scene counts, hook/main/ending structure for 24 seconds, long-form structure behind the unresolved minimum-duration setting, editable draft, versioning, explicit approval, regeneration confirmation, and stale dependency state. Keep user text safe.
```

Phase 4 — AI execution/API Switcher

```
Implement or integrate a single shared AI execution service for all AI tasks. Classify quota, rate limit, transient, invalid-credential, permission, malformed-request and unknown failures. Use eligible failover only, bounded retries, cooldown, job IDs and safe telemetry. Never expose secrets or invent quota metrics. Add provider failure simulations and tests.
```

Phase 5 — Prompt Studio

```
Implement approved-script-to-scene-package generation. Exact Shorts scene counts, 8-second scenes, ratio, detailed visual prompts, narration in selected language, continuity bible, independently copyable complete scene package, editable fields, persistence, single-scene regeneration isolation, and regenerate-all confirmation. Preserve approved script.
```

Phase 6 — Style engine

```
Add cinematic default and the 12 visual styles in the blueprint. Style changes update visual prompt style descriptors and continuity only. Keep script, scene meaning, ratio, duration, and narration unchanged. Protect manually edited prompts and test invariants.
```

Phase 7 — Thumbnail Studio

```
Generate two meaningfully different, detailed thumbnail-image prompts based on approved script, selected mode, language, and ratio. Include composition, focal subject, lighting, color, text-overlay suggestion, realism/style, and negative constraints. Do not generate the image. Save/edit/copy/version and mark stale when script changes.
```

Phase 8 — SEO + export

```
Implement editable titles, description, tags/keywords and appropriate chapters; tie outputs to script version. Implement Review & Export completeness checks, stale warnings, copy individual/all production package, readable Markdown/text export, optional structured JSON export, and secret exclusion.
```

Phase 9 — hardening

```
Run the full blueprint acceptance suite. Test persistence, scene counts, language/mode, stale outputs, manual-edit safety, API failover, telemetry truthfulness, copy/export, accessibility, reduced motion, and responsive layout. Fix failures and provide a precise completion report; do not claim untested functionality.
```

21. OPEN DECISIONS & FINAL CHECKLIST

Owner decision needed

- Confirm 16:9 long-form minimum duration: original source says 4 minutes in one place and 3 minutes in another. Until confirmed, keep configurable and visibly unresolved.
- Confirm deployment/persistence target: prototype-only browser storage versus durable database/backend. This determines what “saved permanently” can honestly mean.
- Confirm which AI providers/models and API connections are actually configured. The UI must reflect real connections and capabilities.
- Confirm whether SEO Package is a required core stage or an optional stage. It appears in the requested HUD stepper but was not part of the original core story-to-prompt workflow; this blueprint includes it as an additional editable packaging stage.
- Confirm whether long-form scenes should also use 8-second clips or variable scene durations. The original 8-second constraint is explicitly tied to the user's Google Flow Shorts workflow; do not force that limit onto long-form without confirmation.

Final implementation checklist

- Product workflow is the main experience, not an unrelated dashboard.
- All original source/story/script/prompt/style/thumbnail logic is present.
- All seven requested HUD steps exist and reflect real dependency state.
- API Switcher is integrated with real generation requests and safe failover.
- Every manual edit is preserved or explicitly confirmed before replacement.
- All outputs are versioned and stale states are accurate.
- All prompts are independently usable when copied.
- Visual style changes do not alter script or narration.
- Two thumbnail prompts are generated and saved.
- Quota, latency, provider status, and progress are truthful.
- All acceptance tests are executed and reported honestly.

END OF MASTER BLUEPRINT — ILLUMATRIX